

Python — STRING (Part 1)

A string is a sequence of characters used to store text.

Strings are immutable, which means their characters cannot be changed directly.

Strings can be written using single, double, or triple quotes.

1. Creating a String

```
s1 = "Hello Python"      # double quotes
s2 = 'Hello Python'      # single quotes
s3 = """This is a
multi-line string"""     # triple quotes
empty = ''
```

Important: slicing creates a new string. The original string remains unchanged.

2. String Indexing

Every character has an index. Indexing starts from 0.

```
s = "Python"
s[0]   # P
s[1]   # y
s[5]   # n
s[-1]  # n ← last character
s[-2]  # o
```

3. Accessing Characters

```
name = "Jahanur"
print(name[0])
print(name[3])
print(name[-1])
```

4. String Slicing

Syntax: `string[start : stop : step]`

The stop position is not included.

```
s = "Python"
s[0:3]   # "Pyt"
s[3:]    # "hon"
s[:4]    # "Pyth"
s[::2]   # "Pto"
s[::-1]  # "nohtyP"
```

Python — STRING (Part 2)

5. String Length

```
s = "Python"
print(len(s))      # 6
```

6. Concatenation & Repetition

```
first = "Hello"
second = "Python"
result = first + " " + second
print(result)      # Hello Python
print("Hi " * 3)   # Hi Hi Hi
```

7. String Membership

```
s = "Python"
print("Py" in s)   # True
print("z" in s)    # False
print("Java" not in s) # True
```

8. Common String Methods ☆

<code>lower()</code>	— convert to lowercase
<code>upper()</code>	— convert to uppercase
<code>title()</code>	— capitalize each word
<code>capitalize()</code>	— capitalize first character
<code>strip()</code>	— remove leading + trailing spaces
<code>lstrip()</code>	— remove leading spaces
<code>rstrip()</code>	— remove trailing spaces
<code>replace(a,b)</code>	— replace substring
<code>split()</code>	— split string into a list
<code>join(iterable)</code>	— join items into a string
<code>find(sub)</code>	— return first index, or -1
<code>count(sub)</code>	— count occurrences
<code>startswith(sub)</code>	— check beginning
<code>endswith(sub)</code>	— check ending

Python — STRING (Part 3)

9. Examples of String Methods

```
s = " Hello Python "
```

s.lower()	# " hello python "
s.upper()	# " HELLO PYTHON "
s.strip()	# "Hello Python"
s.replace("Python", "World")	
s.split()	# ["Hello", "Python"]
"-".join(["A", "B", "C"])	# "A-B-C"

10. Searching in a String

```
s = "I love Python"
```

s.find("Python")	# 7
s.find("Java")	# -1
s.count("o")	# 2
s.startswith("I")	# True
s.endswith("Python")	# True

11. Checking String Content

"Python".isalpha()	# True
"123".isdigit()	# True
"abc123".isalnum()	# True
" ".isspace()	# True
"Python".islower()	# False
"PYTHON".isupper()	# True

12. Escape Characters

Escape sequences let you place special characters inside strings.

\n	# new line
\t	# tab
\"	# double quote
\'	# single quote
\\	# backslash

13. f-Strings ☆

f-strings are a clean way to insert variables into text.

```
name = "Jahanur"
age = 22
print(f"My name is {name} and I am {age}.")
```

f-strings are extremely useful in real Python projects, APIs, logging, and data/ML code.

Python — STRING (Part 4)

14. Strings Are Immutable

You cannot change one character of an existing string directly.
Instead, create a new string.

```
s = "Python"
# s[0] = "J"          × TypeError
s = "J" + s[1:]      # "Jython" ✓
```

15. String Formatting

```
name = "Jahanur"
score = 95
print(f"{name} scored {score}")

price = 99.5
print(f"Price: ₹{price:.2f}")
```

16. Important Points — Quick Revision

- Strings store text.
- Strings are ordered sequences of characters.
- Indexing starts at 0.
- Negative indexing starts at -1.
- Slicing uses [start:stop:step].
- Strings are immutable.
- + joins strings; * repeats strings.
- in / not in check membership.
- Methods such as split(), join(), replace(), find() are very useful.
- f-strings make formatted output simple.

17. Practice Programs ?

1. Reverse a string without using [::-1].
2. Count vowels and consonants.
3. Check whether a string is a palindrome.
4. Count the frequency of each character.
5. Find the first non-repeating character.
6. Remove spaces from a string.
7. Count words in a sentence.
8. Find the longest word in a sentence.
9. Check whether two strings are anagrams.
10. Convert a sentence to title case without title().

AI/ML connection: text data is represented as strings before tokenization, preprocessing, embeddings, and NLP/LLM processing.

Practice every concept with code — understanding comes from doing.